



Silver Box Testing for Scaling Software Development Fidelity: Enhancing Quality and Efficiency

Natali Levi-Soskin and Naomi Unkelos-Shpigel^(✉)

Department of Software Engineering, Braude College of Engineering, Karmiel, Israel
{natalyl1, naomius}@braude.ac.il

Abstract. This study presents Silver Box, a novel software development process that blends white-box, black-box, and gray-box testing into a unified, iterative framework. Designed for cloud-based projects, Silver Box emphasizes continuous testing alongside empathy-driven design thinking to improve software quality. The method was applied in a Cloud Computing course at Braude College with 158 third-year Software Engineering students. Teams developed large-scale systems, beginning with persona interviews and empathy mapping, followed by functional requirements and use case modelling. Despite identical tasks, teams produced diverse use case diagrams, highlighting inconsistent interpretation of requirements. Silver Box integrates early code review and acceptance testing strategies, leading to the detection of new and surprising types of errors—both in students' own code and while executing acceptance tests for other teams' projects. These findings show that cross-project testing fosters deeper understanding and better error identification. Silver Box offers an effective framework for enhancing cloud-native software development and education.

Keywords: software engineering · error detection · software testing · software fidelity

1 Introduction

The growing complexity of software systems necessitates robust testing methodologies that can effectively identify defects and ensure quality. Traditional testing approaches such as Black Box, White Box, and Gray Box each offer distinct advantages but also possess inherent limitations. Black box testing focuses solely on functional requirements without knowledge of the internal code, while White Box testing relies on complete access to the source code to validate internal logic. Gray Box testing attempts to balance these perspectives by leveraging partial knowledge of the system. However, these approaches may overlook critical discrepancies arising from differences in requirement interpretation, creativity in implementation, and edge case handling. To address these gaps, we propose a novel testing methodology termed Silver Box testing. This method involves independent groups developing the same feature or functionality separately, followed by cross-testing, including acceptance tests and code reviews. By introducing

this layer of peer evaluation from an independently developed perspective, Silver Box testing aims to uncover errors and omissions that traditional methods might miss, ultimately improving software quality and aligning development more closely with user expectations.

2 Related Work

As the software industry has evolved, numerous methodological approaches have been formulated. One of the earliest and most traditional approaches, the Waterfall model follows a sequential, linear process with distinct phases such as requirements gathering, design, implementation, testing, and maintenance. Each phase must be completed before moving to the next [1].

To define and develop systems across various domains, the International Council on Systems Engineering (INCOSE) has adopted the Vee model. This model's left branch focuses on decomposing requirements and creating system specifications, whereas the right branch concentrates on integrating parts and validating the system's performance [2]. In addition to the Vee model, another structural mode was defined – the Spiral model. This model involves a cyclical process of planning, risk analysis, development, and evaluation, with each cycle building upon the previous one [3].

Agile methodologies, such as Scrum, Kanban, Extreme Programming (XP), and Lean Software Development, have become widely embraced in recent years. These methodologies emphasize collaboration, iterative and incremental development, frequent feedback loops, and the ability to adapt to changing requirements [4].

While the above methods are software-focused, Model-Driven Development (MDD) uses models and domain-specific languages to automatically generate code, documentation, and other artifacts, aiming to boost productivity and reduce manual work. [5, 6]. Through the application of the Model Fidelity Hierarchy [7], the expectation is that the developed software will be precise. At the lowest level of this hierarchy lies spoken language, followed by informal written text, conceptual models, their enhancement with computational functionalities, and ultimately, an executable iteration of the model. These four levels reveal four types of errors: unexpected values errors, formulae and parameters errors, logical errors, and incomplete information or contradiction errors.

All mentioned approaches share a common thread: the development process [8] involves gathering requirements, modelling the prospective system, developing the software, and ultimately testing it. Despite the variety of testing methods, they all share the same goal: running a program to find errors. [9].

According to the international standard ISO 29119 [10], the testers can either be (1) the developers of the system for removing defects and increasing code quality, (2) the testing group that will create better test cases, (3) the managers for deciding when to stop testing, or (4) the users that will benefit from higher product quality [10]. All types of testers share a common objective: to enhance the system's quality and realize the benefits associated with its effective implementation. In addition, ISO 29119 defines four main types of testing: unit testing, integration testing, system testing, and acceptance testing. All tests are conducted following the code review process, which affects significantly on the defect removal effectiveness [11].

During various stages of software testing and review, different types of errors can be identified. The main motivation of modern code review is improving the quality of the software [12, 13], in addition to an early fix of defects during the development cycle [14]. Unit testing primarily focuses on verifying the correctness of individual units of code [15]. Errors commonly revealed during unit testing include logic errors within the code, boundary condition errors and interface errors between units [16]. Integration tests verify interactions between different units or components of the software [17, 18]. Errors typically revealed during integration tests include interface mismatches, incompatible dependencies between components and data flow errors across integrated modules [19]. When the integration tests are completed, system tests should be performed for assessing the behavior and functionality of the entire software system [20, 21]. Errors commonly revealed during system testing are discrepancies between system requirements and implementation, performance bottlenecks and security vulnerabilities [22]. Finally, after the software development process is completed, acceptance tests validate whether the software meets the requirements and expectations of end-users [23–25]. At this stage, typical errors are usability issues or understanding that some requirements were misinterpreted [26, 34].

In practical terms, the complete collection of test cases is often deemed infinite, making it theoretically impossible to cover all scenarios, even for the most straightforward programs. Consequently, executing such exhaustive testing could demand an extensive amount of time. Therefore, the challenge becomes selecting the most appropriate subset of test cases. Variety of techniques are employed for this purpose, some rooted in risk analysis and others relying on the expertise of test engineers [27].

Three methods of testing applied on a developed software: White box testing, employed to uncover bugs throughout the development phase [27], Black box testing, utilized after the development process is finalized and the product is ready, relying solely on output requirements [28], and Gray box testing, represents a fusion of Whitebox and Black box methodologies, occurs once the product is prepared, incorporating partial knowledge of the internal structure and code [29].

3 Method

In this section, we introduce the research participants, setting, tools, and the testing process.

3.1 Research Participants and Course Description

The research was performed in a third-year course “Introduction to Cloud Computing”, in a bachelor’s degree program in Software Engineering at a College of Engineering. This study included 158 students, in two different semesters, having the exact same task. At the first semester the task was building a game for learning Python while at the second semester the task was building a basic search engine with manager desktop and some statistics. As part of this course, students were required to develop a high-scale software project during the semester, in groups of four to six participants. The project was divided into five stages, each with its own specific purpose.

At stage one, each group had to perform a design thinking, which is used among software engineering practitioners [33], starting with a client interview. After the interview, the rest of the group had to build an empathy map [30, 33] in which they described what the persona does, feels, thinks and says during the interview (see Fig. 1).

DOES: The persona uses popular search engines (like Google). The persona relies on forums and relevant internet communities. The persona relies on his friends for work.	THINKS: The persona hopes that the search engine will understand his phrasing and language and provide accurate results. The persona hopes that the interface will be intuitive and personalized. The persona wonders if the engine will be suitable for his specific needs. The persona wonders if the results will remain up-to-date over time. Samuel is concerned about the reliability of the engine.
SAYS: The persona says, "It is important for me to get relevant search results and an intuitive interface to navigate different topics." "I often struggle to find specific information in guides regarding the requirements of my project." "I would love to receive recommendations and customizations based on my previous searches; it would make me work faster."	FEELS: The persona feels frustrated when the information he finds is not helpful or relevant to his task. The persona fears that he was not understood in presenting the problems and difficulties he is experiencing. The persona is curious about the search engine we intend to develop. The persona feels that if he does not know his job well enough, he will get fired.

Fig. 1. An example of an empathy map

Having the final solution the group members wrote a list of functional and non-functional requirements [31]. Finally, based on the requirements lists and the design thinking process, the groups built a use case diagram [32] for the proposed system.

Considering that all groups had the same task description and underwent the same design process simultaneously, the expectation was to produce similar use case diagrams. However, in practice, the diagrams exhibited differences, some of which were significant.

Figure 2 and Fig. 3 show an example of the differences produced by different groups of students in the search engine and python learning game project. These differ in the number of actors and the purpose of the system.

At stage two the groups defined the future acceptance tests to be performed on their own developed software. The third stage was the software development process. The

fourth stage focused on conducting a code review and performing software testing, and the final, fifth stage was dedicated to performing acceptance tests for their program.

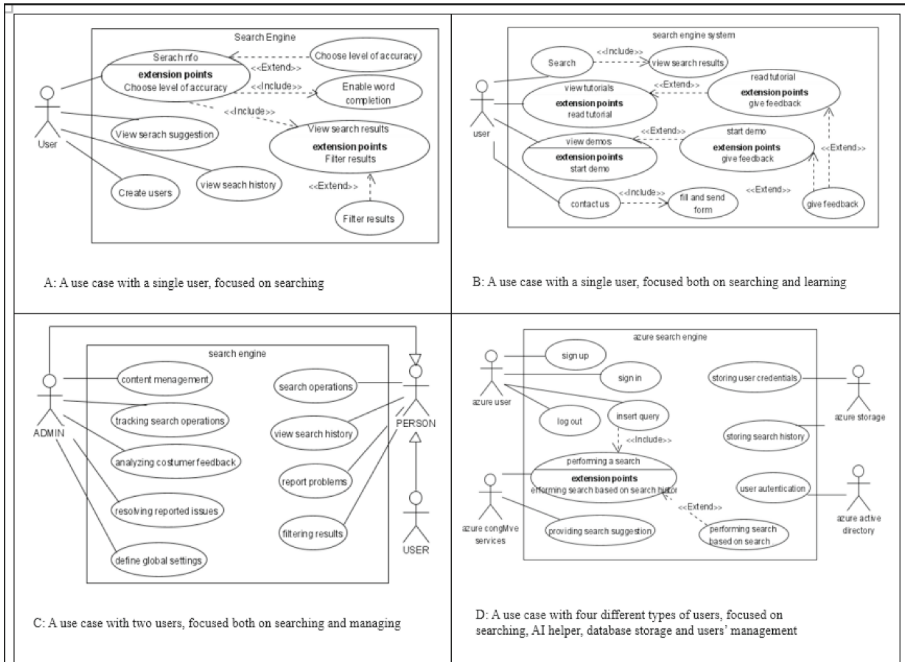


Fig. 2. Four different use cases of the basic search engine task

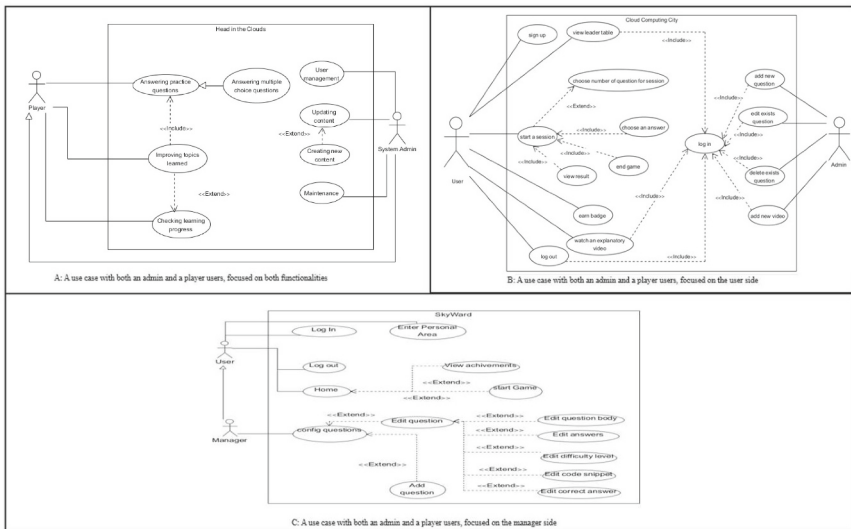


Fig. 3. Use case of the basic Python learning game task

3.2 Special Characteristics of the Second Semester

In the second semester, two additional stages were incorporated into the process. The sixth stage entailed each group executing an acceptance test originally authored by a different group, while the seventh stage involved conducting a code review of another group's implementation.

The integration of these stages facilitated the identification of further errors and contributed to the refinement of the user interface design.

4 Results

4.1 Acceptance Tests

Each group conducted acceptance tests on the system developed by another group. Despite differences in project domains and implementations, most tests passed successfully, indicating that core functionalities were maintained throughout the development lifecycle. In total, 131 acceptance tests were performed, of which 98 passed, resulting in a success rate of approximately 74.8%. While this relatively high success rate reflects a shared understanding of functional requirements and consistent implementation, the fact that 25.2% of the tests failed highlights the added value of the peer testing approach. These failures reveal a non-negligible number of errors that may not have been identified otherwise, emphasizing the importance of structured cross-group validation in enhancing system robustness and quality.

4.2 Code Review

In addition to acceptance testing, each group conducted a code review of a project developed by another team. The reviews focused on code readability, adherence to design principles, modularity, and maintainability. Most reviews identified a range of issues - such as unclear variable naming, duplicate logic, or lack of separation of concerns - which were not detected through functional testing alone. Several groups implemented improvements following the reviews, which led to more structured, maintainable, and consistent codebases. The process also enhanced students' awareness of good programming practices and design considerations. Overall, the code reviews served as an effective tool not only for error detection but also for promoting higher coding standards and mutual learning across groups.

5 Silver Box Testing

In this testing approach, the software is evaluated by an individual or a group assigned to perform the exact same task. Since different individuals may interpret customer requirements differently, vary in their level of creativity in user interface design, or consider different edge cases, testing conducted by someone who completed the task independently can reveal errors or omissions that might not have been identified otherwise. Table 1 gives a summary of the difference between three existing testing approaches and the novel silver box approach.

Table 1. Difference between testing approaches

Criterion	Black Box	White Box	Gray Box	Silver Box
Knowledge of Code	None	Full	Partial	Partial – the tester implemented the same functionality independently. Reviews another group's code
Performed By	External tester (QA)	The developer who wrote the code	Typically, another developer or tester with partial access to the code	A group that developed the same feature independently, testing the work of another group
Focused of Testing	Functionality and user requirements	Functionality and user requirements	Understanding of user requirements, implementation differences, edge cases, and UI design choices	Understanding of user requirements, implementation differences, edge cases, and UI design choices
Perspective	Neutral perspective; simulates end-user behavior	Neutral perspective; simulates end-user behavior	Balanced coverage of functional and structural aspects	Reveals gaps in interpretation, creativity, and hard-to-spot edge cases
Risks	May not cover all code paths	May overlook usability or requirements-related issues	Dependent on limited internal knowledge	Requires redundant development efforts (two groups per feature); needs coordination between groups
Phase in Development Cycle	Post-implementation	During or immediately after coding	Usually later stages, such as integration testing	Post-implementation, after each group has independently completed the feature

6 Summary and Conclusion

Silver Box testing represents an innovative extension to conventional software testing paradigms. By leveraging parallel independent development and subsequent cross-validation, it effectively uncovers subtle bugs and design flaws that arise from varying interpretations of user requirements and creative differences in implementation. This approach also facilitates a more comprehensive evaluation of the user interface and edge case scenarios, areas where other methods may lack depth. Although Silver Box testing requires additional development resources due to the need for duplicate implementations, the benefits include improved defect detection and enhanced collaboration among

development teams. The method's capacity to expose discrepancies and promote rigorous code review can lead to higher software reliability and user satisfaction. Future work should explore optimizing Silver Box testing to balance its resource demands with its quality assurance benefits, potentially integrating automated tools to streamline acceptance testing and code review processes. Overall, Silver Box testing offers a promising complement to existing methodologies, particularly for projects demanding high robustness and nuanced understanding of requirements.

References

1. Royce, W.W.: Managing the Development of Large Software Systems (1970) (2021)
2. Li, L., Levi Soskin, N., Jbara, A., Karpel, M., Dori, D.: Model-based systems engineering for aircraft design with dynamic landing constraints using object-process methodology. *IEEE Access* (2019)
3. Boehm, B.W.: A spiral model of software development and enhancement. *Computer* (Long Beach Calif) **21**(5), 61–72 (1988)
4. Ambler, S.W., Lines, M.: Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise. IBM press (2012)
5. Hailpern, B., Tarr, P.: Model-driven development: the good, the bad, and the ugly. *IBM Syst. J.* **45**(3), 451–461 (2006)
6. Selic, B.: The pragmatics of model-driven development. *IEEE Softw.* **20**(5), 19–25 (2003). <https://doi.org/10.1109/MS.2003.1231146>
7. Levi-Soskin, N., Jbara, A., Dori, D.: The model fidelity hierarchy: from text to conceptual, computational, and executable model. *IEEE Syst. J.* **15**(1), 1287–1298 (2020)
8. Parezanovic, N.: *Racunarstvo i informatika. Zavod za udzbenike i nastavna sredstva*–Beograd (1996)
9. Abran, A., Buglione, L., Sellami, A.: Software Measurement Body of Knowledge & Initial Validation using Vincenti's Classification of Engineering Knowledge types (2004)
10. Iso. Software and Systems Engineering-Software Testing-Part 1: General Concepts (2022). Accessed: May 13, 2024. <https://www.iso.org/standard/81291.html>
11. Kemerer, C.F., Paulk, M.C.: The impact of design and code reviews on software quality: an empirical study based on psp data. *IEEE Trans. Software Eng.* **35**(4), 534–550 (2009)
12. Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 712–721 (2013)
13. McIntosh, S., Kamei, Y., Adams, B., Hassan, A.E.: An empirical study of the impact of modern code review practices on software quality. *Empir. Softw. Eng.* **21**, 2146–2189 (2016)
14. Fagan, M.E.: Design and code inspections to reduce errors in program development. *IBM Systems Journal* **38**(2.3), 258–287 (1999)
15. Daka, E., Fraser, G.: A survey on unit testing practices and problems. In: 2014 IEEE 25th International Symposium on Software Reliability Engineering, pp. 201–211 (2014)
16. Binder, R.: *Testing Object-Oriented Systems: Models, Patterns, and Tools*. Addison-Wesley Professional (2000)
17. Leung, H.K.N., White, L.: A study of integration testing and software regression at the integration level. *Proceedings. Conference on Software Maintenance* **1990**, 290–301 (1990)
18. Jorgensen, P.C., Erickson, C.: Object-oriented integration testing. *Commun. ACM* **37**(9), 30–38 (1994)
19. Kaner, C.: Software engineering metrics: what do they measure and how do we know?, In: *Proc. Int'l Software Metrics Symposium*, Chicago, IL, USA, Sept. 2004, pp. 1–12 (2004)

20. Hooda, I., Chhillar, R.S.: Software test process, testing types and techniques. *Int J Comput Appl.* **111**(13) (2015)
21. Simpson, W.R., Sheppard, J.W.: *System Test and Diagnosis*. Springer Science & Business Media (2012)
22. Pressman, R.S., et al.: A practitioner's approach. *Software Engineering* **2**, pp. 41–42 (2010)
23. Davis, F.D., Venkatesh, V.: Toward preprototype user acceptance testing of new information systems: implications for software project management. *IEEE Trans. Eng. Manag.* **51**(1), 31–46 (2004)
24. Ricca, F., Torchiano, M., Di Penta, M., Ceccato, M., Tonella, P.: Using acceptance tests as a support for clarifying requirements: a series of experiments. *Inf. Softw. Technol.* **51**(2), 270–283 (2009)
25. Leung, H.K.N., Wong, P.W.L.: A study of user acceptance tests. *Software Qual. J.* **6**, 137–149 (1997)
26. Cohn, M.: *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional (2004)
27. Jovanović, I.: Software testing methods and techniques. *The IPSI BgD Transactions on Internet Research* **30** (2006)
28. Steegmans, E., et al.: Black & white testing: Bridging black box testing and white box testing. *Software Testing: Beheers Optimaal de Risico's van IT in uw Business*, pp. 1–12 (2004)
29. Tonella, P., Ricca, F.: A 2-layer model for the white-box testing of web applications. In: *Proceedings. Sixth IEEE International Workshop on Web Site Evolution*, pp. 11–19 (2004)
30. Ferreira, B., Silva, W., Oliveira, E., Conte, T.: Designing personas with empathy map. In: *SEKE* (2015)
31. Zhou, J.Y.: *Functional Requirements and Non-Functional Requirements: A Survey*. Concordia University (2004)
32. U. M. L. R. T. Force. *OMG Unified Modeling Language Specification*, version 1.3, June 1999 (1999)
33. Unkelos-Shpigel, N.: Heights of glory: design thinking to enhance battle monuments visits via augmented reality. *Proceedings of AVICH* (2024)
34. Unkelos-Shpigel, N., Sheidin, J., Kupfer, M.: Climb your way to the model: Teaching UML to software engineering students. In: *International Conference on Advanced Information Systems Engineering*, pp. 40–46. Springer (2019)